

1804.10694v5: formula evidence

The page, the confidence and the picture of an inline formula are its HOST LINE’s — a formula has none of its own. A line’s confidence is not a formula’s.

Inline formulas (first occurrence)

Identifier	Page	Conf.	LaTeX source	Rendered	Image
1804.10694v5_F00001	1	■ 1.000	$C=\alpha A B+\beta C$	$C=\alpha AB+\beta C$	(gemm), which computes $C=\alpha AB+\beta C$ and is a building
1804.10694v5_F00002	2	■ 0.995	$2.3\times$	$2.3\times$	$2.3\times$. We also evaluate TIRAMISU on a set of image
1804.10694v5_F00003	4	■ 1.000	32×32	32×32	splitting, shifting, etc. For example, applying 32×32 loop
1804.10694v5_F00004	4	■ 0.560	$\mathbf{C}$	C	We assume that C and P are computations, b is a buffer
1804.10694v5_F00005	4	■ 0.560	$\mathbf{P}$	P	We assume that C and P are computations, b is a buffer
1804.10694v5_F00006	4	■ 0.560	$\mathbf{b}$	b	We assume that C and P are computations, b is a buffer
1804.10694v5_F00007	4	■ 0.999	(i,j)	(i,j)	Tile the loop levels (i,j) of the computation C
1804.10694v5_F00008	4	■ 1.000	$t_1\times t_2$	$t1\times t2$	by $t1\times t2$. The names of the new loop levels
1804.10694v5_F00009	4	■ 1.000	P	P	Compute the computation P in the loop nest of $\bar{C}$
1804.10694v5_F00010	4	■ 1.000	C	C	Compute the computation P in the loop nest of $\bar{C}$
1804.10694v5_F00011	5	■ 0.374	$((\mathrm{N}-2)\ \% 32,32)+2)$	$((N-2)\%32,32)+2)$	GPUThread for (i1 in 0.. $\min((N-2)\%32,32)+2)$
1804.10694v5_F00012	5	■ 0.725	$((\mathrm{M}-2)\ \% 32,32)+2)$	$((M-2)\%32,32)+2)$	GPUThread for (j1 in 0.. $\min((M-2)\%32,32)+2)$
1804.10694v5_F00013	6	■ 0.817	j_0	$j0$	by by at the loop level $j0$. This transformation introduces re-
1804.10694v5_F00014	6	■ 0.943	$[\mathrm{n}*((\mathrm{N}-2)/$	$[n*((N-2)/$	$in[n*((N-2)/Nodes)..(n+1)*((N-2)/Nodes),*,*]$
1804.10694v5_F00015	6	■ 0.943	$)\ldots(\mathrm{n}+1)*((\mathrm{N}-2)/$	$)...(n+1)*((N-2)/$	$in[n*((N-2)/Nodes)..(n+1)*((N-2)/Nodes),*,*]$
1804.10694v5_F00016	6	■ 0.943	$),*,*]$	$),*,*]$	$in[n*((N-2)/Nodes)..(n+1)*((N-2)/Nodes),*,*]$
1804.10694v5_F00017	6	■ 0.860	$\mathrm{n}*(\mathrm{N}-2)/$	$n*((N-2)/$	In other words, this corresponds to row $n*(N-2)/Nodes$
1804.10694v5_F00018	6	■ 0.914	$(\mathrm{n}+1)*((\mathrm{N}-2)/$	$(n+1)*((N-2)/$	through row $(n+1)*((N-2)/Nodes)$. This chunk is stored
1804.10694v5_F00019	6	■ 0.914	$)$	$)$	through row $(n+1)*((N-2)/Nodes)$. This chunk is stored
1804.10694v5_F00020	6	■ 0.982	$\mathrm{n}+1$	$n+1$	first two rows of data from the chunk stored in node $n+1$. These
1804.10694v5_F00021	6	■ 1.000	$M\times 2\times 3$	$M\times 2\times 3$	will send 2 rows ($M\times 2\times 3$ contiguous data elements) from
1804.10694v5_F00022	6	■ 0.997	$\operatorname{lin}(0,0,0)$	$\operatorname{lin}(0,0,0)$	node is to node is-1 starting from $\operatorname{lin}(0,0,0)$, which
1804.10694v5_F00023	7	■ 1.000	i	i	where i and j are the dimensions of the tuples in the set.
1804.10694v5_F00024	7	■ 1.000	j	j	where i and j are the dimensions of the tuples in the set.
1804.10694v5_F00025	7	■ 1.000	$S_1(i,j)$	$S1(i,j)$	(e.g. the tuple $S1(i,j)$ maps to the tuple $S2(i+2,j+2)$).
1804.10694v5_F00026	7	■ 1.000	$S_2(i+2,j+2)$	$S2(i+2,j+2)$	(e.g. the tuple $S1(i,j)$ maps to the tuple $S2(i+2,j+2)$).
1804.10694v5_F00027	7	■ 1.000	$\{by(i,j,c):0\leq i<N-2\wedge 0\leq j<M-2\wedge 0\leq$	$\{by(i,j,c):0\leq i<N-2\wedge 0\leq j<M-2\wedge 0\leq$	The first part, $\{by(i,j,c):0\leq i<N-2\wedge 0\leq j<M-2\wedge 0\leq$
1804.10694v5_F00028	7	■ 1.000	$c<3\}$	$c<3\}$	$c<3\}$, specifies the iteration domain of the computation
1804.10694v5_F00029	7	■ 0.555	(i,j,c)	(i,j,c)	iteration domain is the set of tuples $by(i,j,c)$ such that
1804.10694v5_F00030	7	■ 1.000	$0\leq i<N-2\wedge 0\leq j<M-2\wedge 0\leq c<3$	$0\leq i<N-2\wedge 0\leq j<M-2\wedge 0\leq c<3$	$0\leq i<N-2\wedge 0\leq j<M-2\wedge 0\leq c<3$. Computations in
1804.10694v5_F00031	7	■ 0.968	$(i/32)\wedge j_0=$	$(i/32)\wedge j0=$	$\text{floor}(i/32)\wedge j0=\text{floor}(j/32)\wedge i1=i\%32\wedge j1=j\%32\wedge 0\leq i<$
1804.10694v5_F00032	7	■ 0.968	$(j/32)\wedge i_1=i\%32\wedge j_1=j\%32\wedge 0\leq i<$	$(j/32)\wedge i1=i\%32\wedge j1=j\%32\wedge 0\leq i<$	$\text{floor}(i/32)\wedge j0=\text{floor}(j/32)\wedge i1=i\%32\wedge j1=j\%32\wedge 0\leq i<$
1804.10694v5_F00033	7	■ 1.000	$N-2\wedge 0\leq j<M-2\wedge 0\leq c<3\}:(bx(i0*32+i1,j0*32+j1,c)+$	$N-2\wedge 0\leq j<M-2\wedge 0\leq c<3\}:(bx(i0*32+i1,j0*32+j1,c)+$	$N-2\wedge 0\leq j<M-2\wedge 0\leq c<3\}:(bx(i0*32+i1,j0*32+j1,c)+$
1804.10694v5_F00034	7	■ 1.000	$bx(i0*32+i1+1,j0*32+j1,c)+bx(i0*32+i1+2,j0*32+j1,c))/3$	$bx(i0*32+i1+1,j0*32+j1,c)+bx(i0*32+i1+2,j0*32+j1,c))/3$	$bx(i0*32+i1+1,j0*32+j1,c)+bx(i0*32+i1+2,j0*32+j1,c))/3$
1804.10694v5_F00035	7	■ 0.792	g_{puB}	$gpuB$	is an ordered set of computations. The tag $gpuB$ for the
1804.10694v5_F00036	7	■ 1.000	i_0	$i0$	dimension $i0$ and $j0$ indicates that each iteration $(i0,j0)$ is
1804.10694v5_F00037	7	■ 1.000	(i_0,j_0)	$(i0,j0)$	dimension $i0$ and $j0$ indicates that each iteration $(i0,j0)$ is
1804.10694v5_F00038	7	■ 0.973	$S_0(0,0,0)$	$S0(0,0,0)$	² For example the computation $S0(0,0,0)$ is lexicographically before the
1804.10694v5_F00039	7	■ 0.818	$S_0(0,0,1)$	$S0(0,0,1)$	computation $S0(0,0,1)$ and the computations $S0(0,i,0)$ are lexicographically
1804.10694v5_F00040	7	■ 0.818	$S_0(0,i,0)$	$S0(0,i,0)$	computation $S0(0,0,1)$ and the computations $S0(0,i,0)$ are lexicographically
1804.10694v5_F00041	7	■ 1.000	$S_0(1,i,0)$	$S0(1,i,0)$	before the computations $S0(1,i,0)$
1804.10694v5_F00042	8	■ 1.000	$c(i,j)$	$c(i,j)$	complicated accesses such as the storage of computations $c(i,j)$

Identifier	Page	Conf.	LaTeX source	Rendered	Image
1804.10694v5_F00043	8	■ 1.000	$c(i \% 2, j \% 2)$	$c(i\%2, j\%2)$	into the array elements $c(i\%2, j\%2)$ or into $c(j, i)$.
1804.10694v5_F00044	8	■ 1.000	$c(j, i)$	$c(j, i)$	into the array elements $c(i\%2, j\%2)$ or into $c(j, i)$.
1804.10694v5_F00045	8	■ 0.971	$[c, i, j]$	$[c, i, j]$	computation $by(i, j, c)$ is stored in the array element $by[c, i, j]$
1804.10694v5_F00046	9	■ 0.957	1060×1060	1060×1060	use matrices of size 1060×1060 and vectors of size 1060
1804.10694v5_F00047	9	■ 1.000	512×512	512×512	while for Conv and VGG we use 512×512 as the data input
1804.10694v5_F00048	10	■ 0.963	$3 \times 3, 5 \times 5$	$3 \times 3, 5 \times 5$	versions for common convolution filter sizes (3×3 , 5×5 ,
1804.10694v5_F00049	10	■ 0.992	$7 \times 7, 9 \times 9$	$7 \times 7, 9 \times 9$	7×7 , 9×9 and 11×11). This allows the TIRAMISU compiler
1804.10694v5_F00050	10	■ 0.992	11×11	11×11	7×7 , 9×9 and 11×11). This allows the TIRAMISU compiler
1804.10694v5_F00051	10	■ 0.991	$\mathrm{x} \geq$	$x \geq$	but the iteration space is not rectangular (it tests if $x \geq$
1804.10694v5_F00052	10	■ 0.955	$2112 \times 3520 \mathrm{RGB}$	$2112 \times 3520 \mathrm{RGB}$	We used a 2112×3520 RGB input image for the experiments.
1804.10694v5_F00053	10	■ 0.856	≥ 1	≥ 1	cycle length ≥ 1 (R is written in the first statement and read
1804.10694v5_F00054	11	■ 0.989	$3.25 \times$	$3.25 \times$	distributed Halide in each case. It achieves up to $3.25 \times$ speedup
1804.10694v5_F00055	11	■ 0.718	$i < 0, N$	$i < 0, N$	⁴ <code>clamp(i, 0, N)</code> returns 0 if $i < 0$, N if $i > N$, i otherwise.
1804.10694v5_F00056	11	■ 0.718	$i > N, i$	$i > N, i$	⁴ <code>clamp(i, 0, N)</code> returns 0 if $i < 0$, N if $i > N$, i otherwise.